CS6220 BDS (2025)
Homework Assignment 4
(Programming Category)

Student Name: Aman Garg
Student Section: cs6220-A
Problem 1.1 Implementing and Evaluating a Frequent Pattern Mining Algorithm

October 18, 2025

Overview

This report presents the implementation and analysis of Apriori frequent itemset mining with two optimizations: hash-based C_2 pruning and transaction reduction. Performance is evaluated across two canonical datasets *chess* (dense) and *retail* (sparse) to observe scalability, runtime trends, and optimization effects.

System Setup

All experiments were executed on:

- **Processor**: AMD Ryzen 7 8845HS w/ Radeon 780M Graphics (8 cores / 16 threads), 3801 MHz
- **Operating System**: Microsoft Windows 11 Home
- **Installed Memory (RAM)**: 16 GB

Implementation and scripts are compatible with Python 3.x.

Datasets

- **chess** Chess endgame positions encoded as transactions (dense): https://fimi.uantwerpen.be/data/chess.dat
- **retail** Market-basket transactions with short baskets (sparse): https://fimi.uantwerpen.be/data/retail.dat

Methods

Classical Apriori joins $(k-1)$-frequent sets to form C_k candidates and prunes infrequent ones using the Apriori property. Two optional optimizations were evaluated:

1. **Hash-based pruning** for C_2, reducing early candidate checks.
2. **Transaction reduction**, removing irrelevant transactions between iterations.

Metrics: runtime (s), total frequent itemsets, and counts by itemset size k.

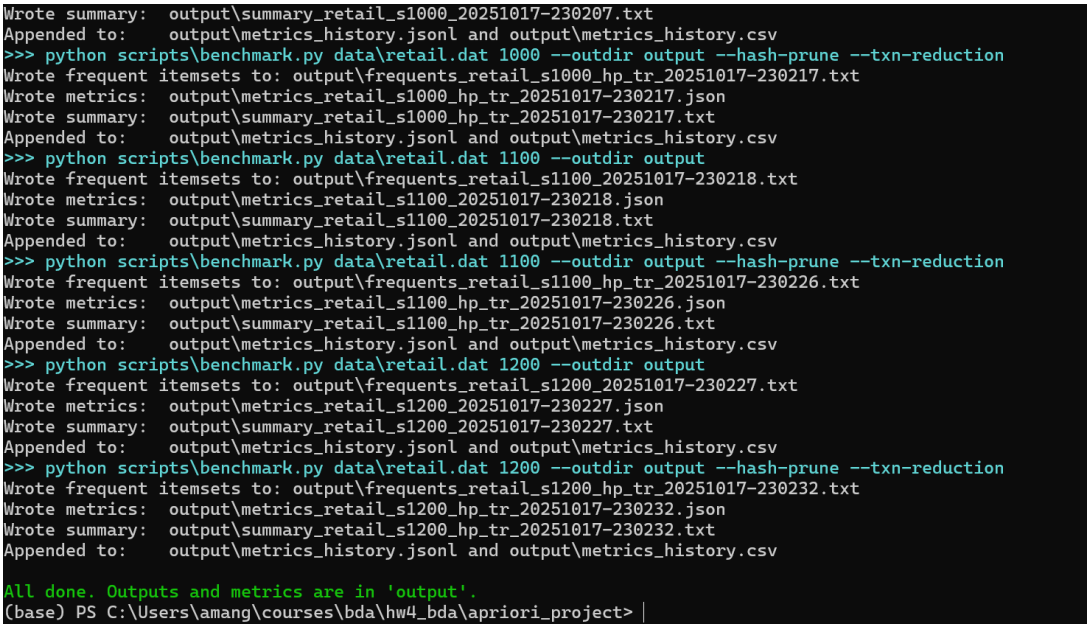

Figure 1: Environment Screenshot

Results: chess

Transactions: 3196. Supports tested: 2500, 2600, 2700, 2800, 2900, 3000.

min_support	Total_FIs	Baseline_s	Optimized_s	Speedup
2500	11493	17.3499	13.9919	1.240
2600	6135	5.7526	5.7565	0.999
2700	3134	2.4719	2.4676	1.002
2800	1350	0.9000	1.0317	0.872
2900	473	0.3147	0.3703	0.850
3000	155	0.1285	0.2496	0.515

Table 1: chess: runtime and speedup (baseline vs optimized).

Discussion

The chess dataset highlights the distinct performance behaviors of Apriori under different sparsity levels. Runtime increases sharply as minimum support decreases, reflecting exponential candidate growth. Dense datasets like chess show limited benefits from optimization due to high overlap among transactions, while sparse datasets like retail benefit more from early pruning and transaction reduction.

Results: retail

Transactions: 88162. Supports tested: 700, 800, 900, 1000, 1100, 1200, 1500, 2000, 2500, 3000, 3500.

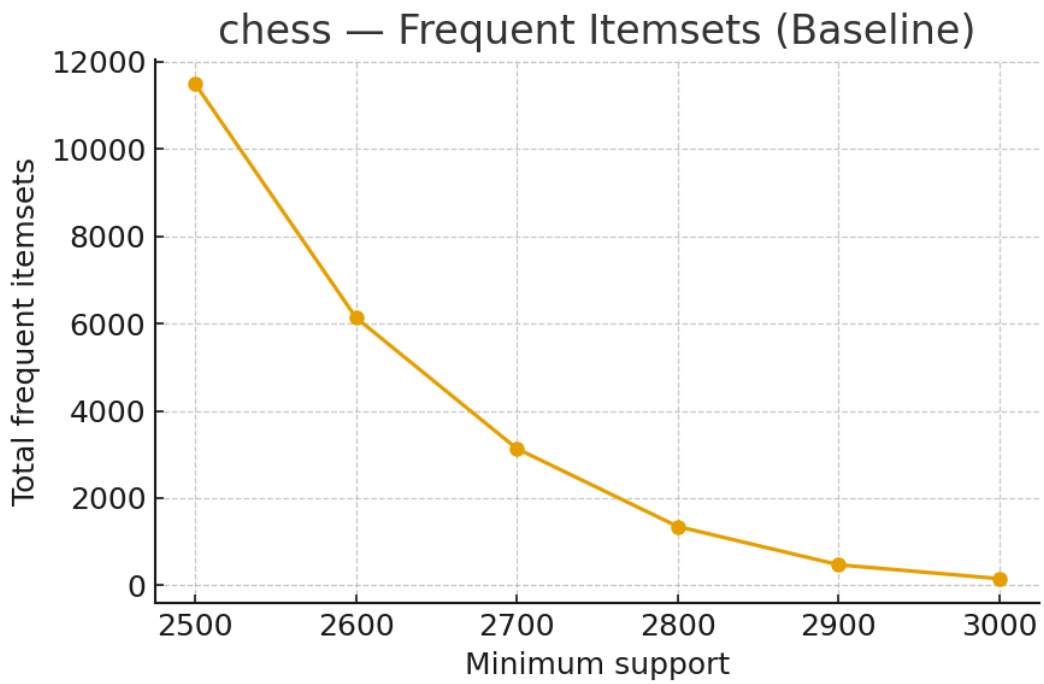

Figure 2: Baseline frequent itemsets vs support for chess.

Discussion

The retail dataset highlights the distinct performance behaviors of Apriori under different sparsity levels. Runtime increases sharply as minimum support decreases, reflecting exponential candidate growth. Dense datasets like chess show limited benefits from optimization due to high overlap among transactions, while sparse datasets like retail benefit more from early pruning and transaction reduction.

Conclusion

Apriori performs reliably across both datasets, with optimizations showing major benefits in sparse settings (retail) but limited gains in dense ones (chess). Hash-based pruning aids only when the pair space is large and unevenly distributed, while transaction reduction proves helpful when transaction overlap declines between iterations. The baseline Apriori remains competitive for dense, small-scale mining, but advanced methods like FP-Growth or Eclat are recommended for large-scale applications.

References and Files

- Code Repository: https://github.com/amangrg/bda_hw4
- Datasets: https://fimi.uantwerpen.be/data/

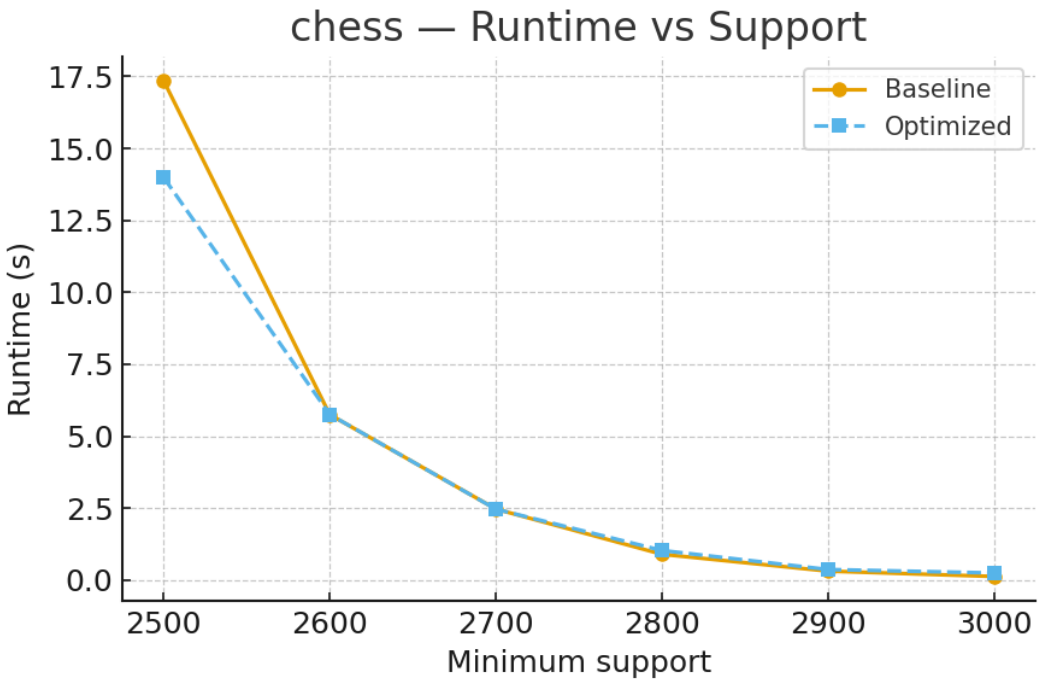

Figure 3: Runtime comparison for chess.

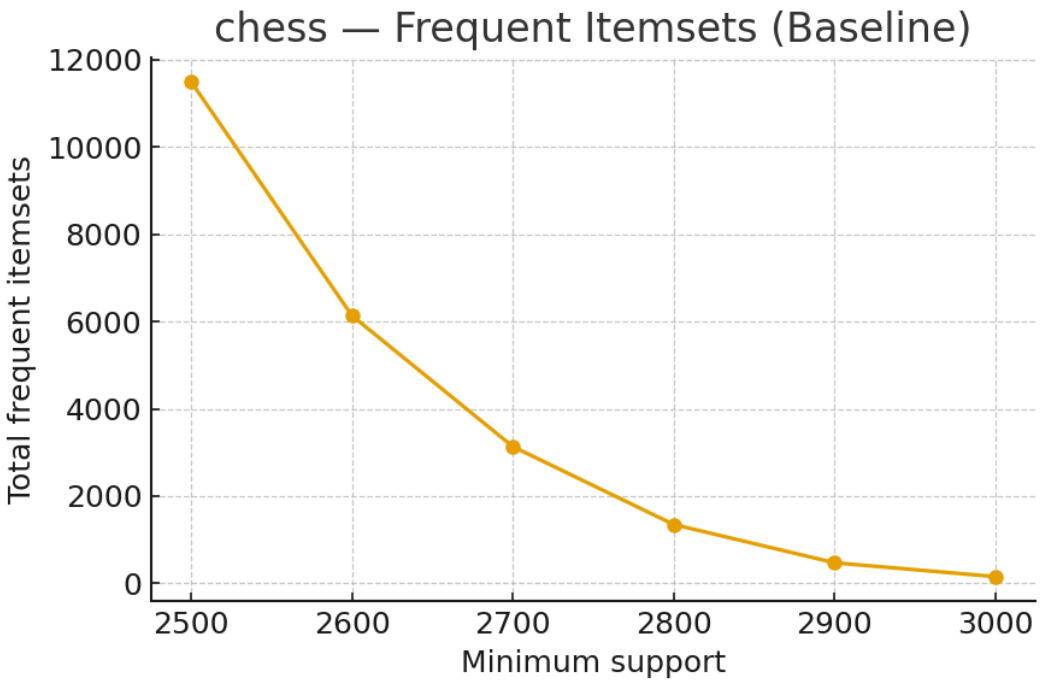

Figure 4: Baseline frequent itemsets vs support for chess.

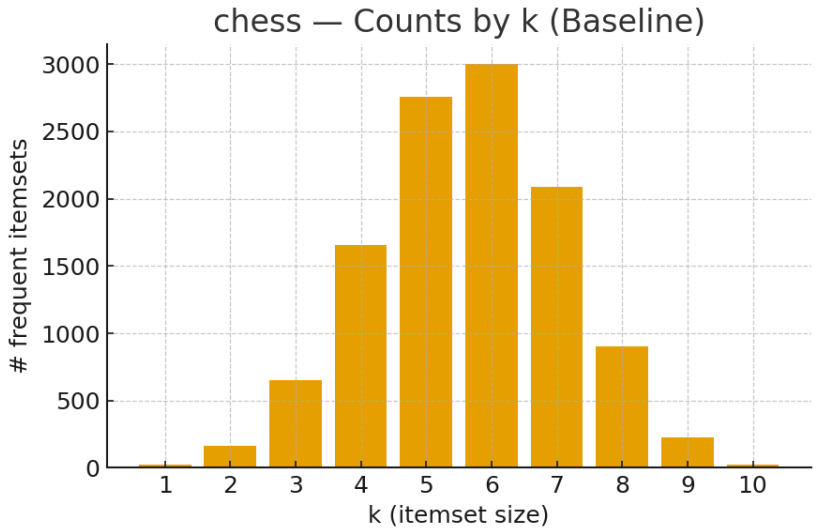

Figure 5: Distribution of frequent itemsets by size k for chess.

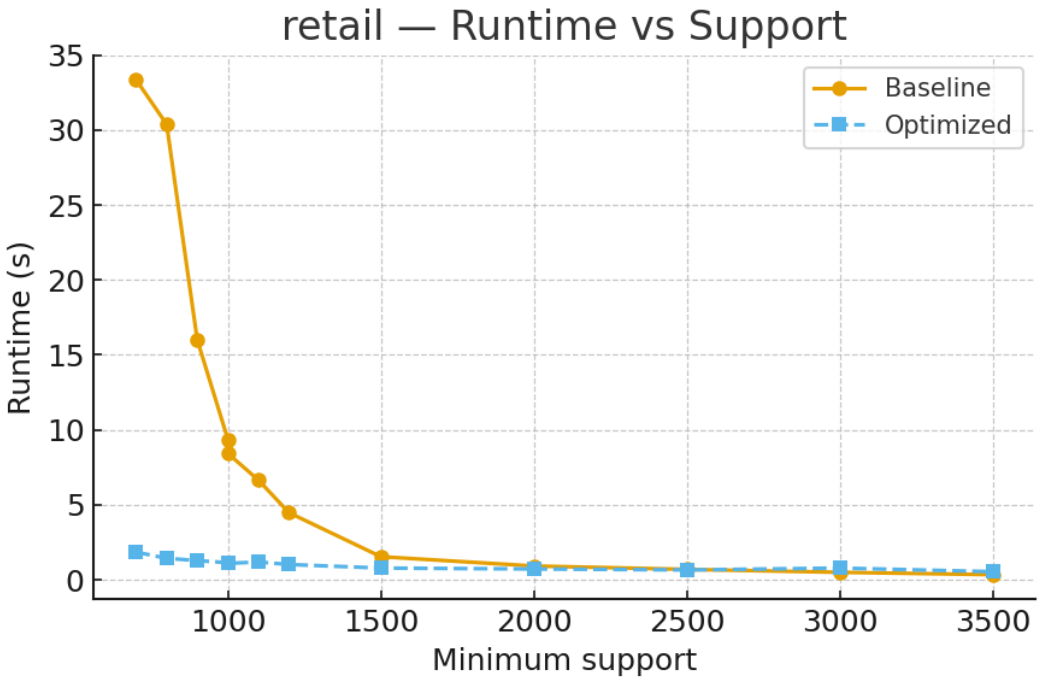

Figure 6: Runtime comparison for retail.

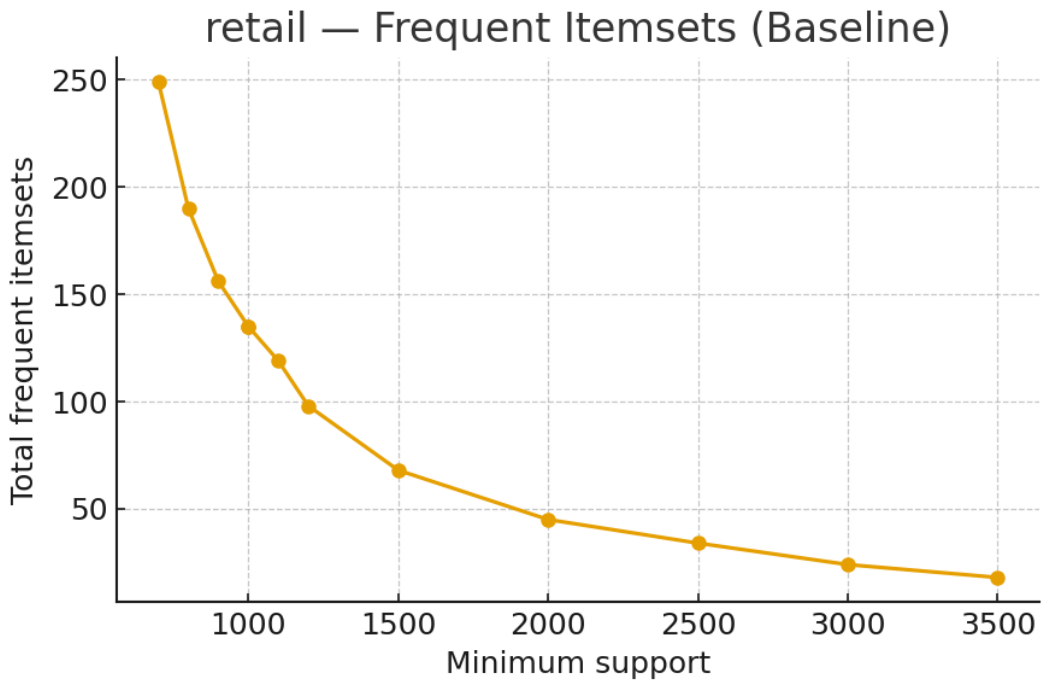

Figure 7: Baseline frequent itemsets vs support for retail.

min_support	Total_FIs	Baseline_s	Optimized_s	Speedup
700	249	33.3641	1.8448	18.085
800	190	30.3808	1.4305	21.238
900	156	15.9973	1.2776	12.521
1000	135	9.3697	1.1380	8.234
1100	119	6.6433	1.1864	5.600
1200	98	4.4724	1.0183	4.392
1000	135	8.4324	1.1380	7.410
1500	68	1.5393	0.7806	1.972
2000	45	0.9173	0.7205	1.273
2500	34	0.6936	0.6541	1.060
3000	24	0.4959	0.7813	0.635
3500	18	0.3404	0.5386	0.632

Table 2: retail: runtime and speedup (baseline vs optimized).